CLASS ACTIVITY

UNIT V

NAME: R. Reddy Balaji

Reg:192424122

Q1. Communication of Rare Surgical Risk

Aim

To evaluate two different ways of communicating a rare surgical complication risk to patients and determine which framing is more understandable for a layperson.

Problem Statement

A hospital communicates a 2% risk of a rare surgical complication in two ways:

- **Version A:** “There is a 2% chance of complication.”
- **Version B:** “Out of 100 patients like you, 2 will experience a complication.”

The objective is to determine which representation is easier for a general patient to understand.

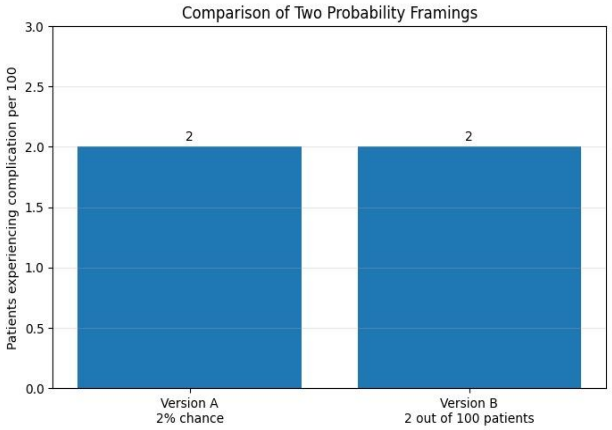
Dataset

Version	Risk Percentage	Patients Considered	Expected Complications
A – Percentage	2%	100	2
B – Natural Frequency	2%	100	2

Code

```
import matplotlib.pyplot as plt


versions = ["Version A\n2% chance", "Version B\n2 out of 100"]

values = [2, 2]


plt.figure(figsize=(7, 5))

bars = plt.bar(versions, values)


plt.ylabel("Expected complications per 100 patients")
```

```
plt.title("Comparison of Risk Communication Formats")
plt.ylim(0, 3)


for bar, value in zip(bars, values):
    plt.text(bar.get_x() + bar.get_width()/2,
             value + 0.05,
             str(value),
             ha="center")


plt.show()
```



Result

Both versions communicate the same numerical probability: **2%**. However, Version B is generally easier for a layperson to understand because it uses a **natural-frequency format**.

The statement “2 out of 100 patients” gives the patient a concrete reference group. In contrast, “2% chance” requires the patient to mentally convert a percentage into a frequency.

Interpretation

A well-known pitfall in probability communication is **difficulty understanding percentages and probabilities**, particularly when people are unfamiliar with statistical concepts. Natural frequencies can make small risks more concrete and easier to compare.

Conclusion

For a general layperson, **“2 out of 100 patients” is likely to be understood more accurately than “2% chance.”** Natural frequencies reduce the mental effort required to interpret a small probability. Therefore, Version B should be the default for patient-facing communication, while Version A can be used in technical or standardized medical contexts.

Q2. Forecast Uncertainty for Tomorrow's Rainfall

Aim

To compare a static confidence-interval band and an animated Hypothetical Outcome Plot (HOP) for communicating rainfall forecast uncertainty to a general public audience on a mobile application.

Problem Statement

A weather agency must choose between:

- **Method A:** Static confidence-interval band.

- **Method B:** Animated Hypothetical Outcome Plot showing simulated rainfall outcomes.

The methods are evaluated based on comprehension speed, risk of misinterpretation, and technical feasibility.

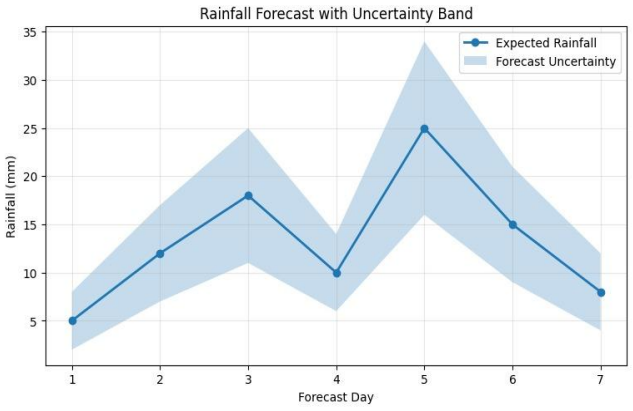
Dataset

The following simulated rainfall forecast data are used:

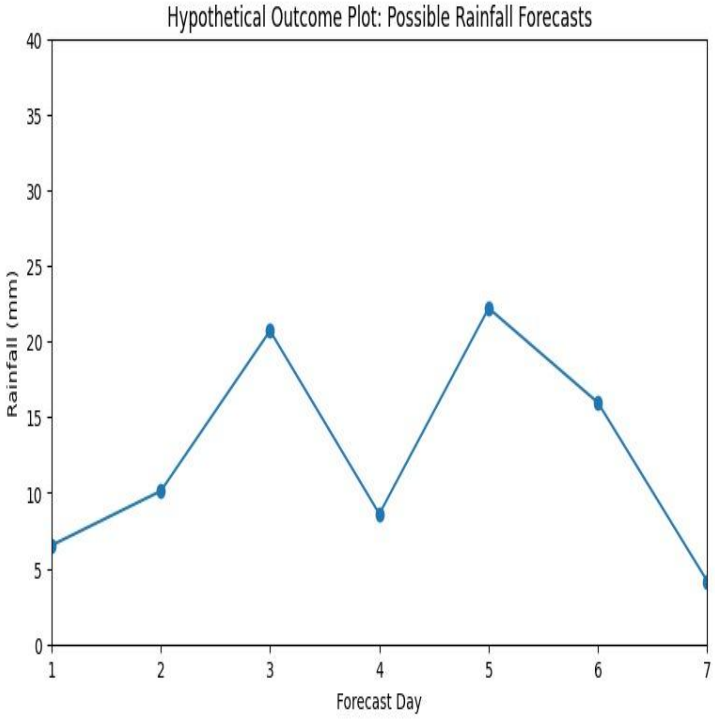
Hour	Forecast Rainfall (mm)	Lower Bound (mm)	Upper Bound (mm)
6	1.5	0.5	3.0
9	2.0	0.8	4.0
12	4.0	1.5	7.0
15	5.0	2.0	9.0
18	3.5	1.0	7.0
21	2.0	0.5	4.5

Code

```
import numpy as np

import matplotlib.pyplot as plt


hours = np.array([6, 9, 12, 15, 18, 21])

forecast = np.array([1.5, 2.0, 4.0, 5.0, 3.5, 2.0])

lower = np.array([0.5, 0.8, 1.5, 2.0, 1.0, 0.5])

upper = np.array([3.0, 4.0, 7.0, 9.0, 7.0, 4.5])


plt.figure(figsize=(8, 5))


plt.plot(hours, forecast, marker="o", label="Forecast")

plt.fill_between(hours, lower, upper, alpha=0.3,

                 label="Uncertainty interval")


plt.xlabel("Time of Day")

plt.ylabel("Rainfall (mm)")
```



plt.title("Tomorrow's Rainfall Forecast Uncertainty")



plt.legend()

plt.grid(True, alpha=0.3)

plt.show()

Result

The static uncertainty band presents the forecast and its uncertainty in a single view. Users can quickly identify the expected rainfall and the range of possible outcomes.

An animated HOP can communicate uncertainty intuitively by repeatedly showing simulated possible rainfall outcomes. However, users must watch the animation to appreciate the **Interpretation**

For a general public mobile application, speed and simplicity are important. A static uncertainty band allows users to understand the forecast immediately without waiting for an animation.

Q3. Startup Revenue Growth and Bar Chart Design

Aim

To evaluate a startup revenue chart with respect to proportional ink, baseline selection, and axis scale, and determine whether correcting the chart strengthens the credibility of the growth story.

Problem Statement

A startup's revenue increases from ₹2 lakhs to ₹2 crores over three years. The original bar chart uses a linear axis with a non-zero baseline, causing the final bar to appear disproportionately large.

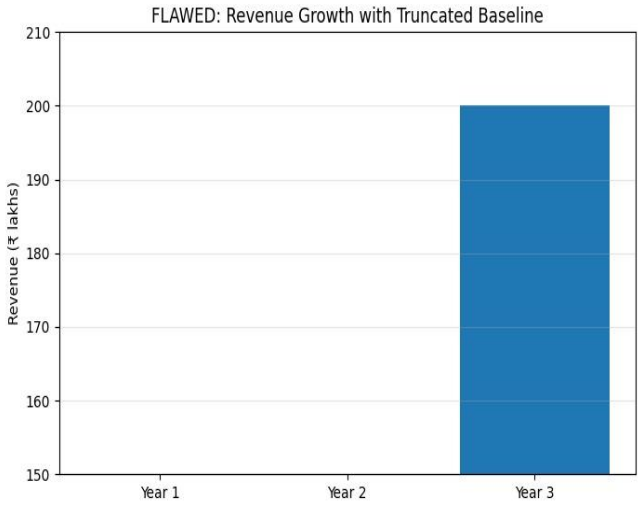
Dataset

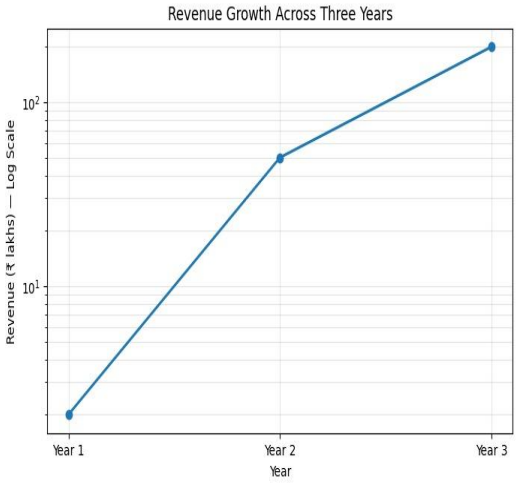
Year	Revenue (₹ Lakhs)	Revenue (₹ Crores)
Year 1	2	0.02
Year 2	10	0.10
Year 3	200	2.00

Code

```
import matplotlib.pyplot as plt

years = ["Year 1", "Year 2", "Year 3"]
revenue = [2, 10, 200]

# Original chart with misleading baseline
plt.figure(figsize=(7, 5))
plt.bar(years, revenue, bottom=1)

plt.ylabel("Revenue (₹ Lakhs)")
plt.title("Original Revenue Chart with Non-Zero Baseline")
plt.show()

# Corrected chart
plt.figure(figsize=(7, 5))
plt.bar(years, revenue)

plt.ylabel("Revenue (₹ Lakhs)")
plt.title("Corrected Revenue Chart with Zero Baseline")
plt.show()
```

v





Result

The original chart exaggerates the visual difference between revenue values because the baseline does not start at zero.

A corrected bar chart beginning at zero represents the magnitude of each bar proportionally.

Interpretation

Bar charts rely heavily on the length of the bars to represent numerical magnitude. Therefore, a truncated baseline can exaggerate differences and violate the principle of **proportional ink**, where the visual amount used to represent data should correspond appropriately to the actual numerical values.

The corrected chart may initially appear less dramatic because the visual exaggeration has

.

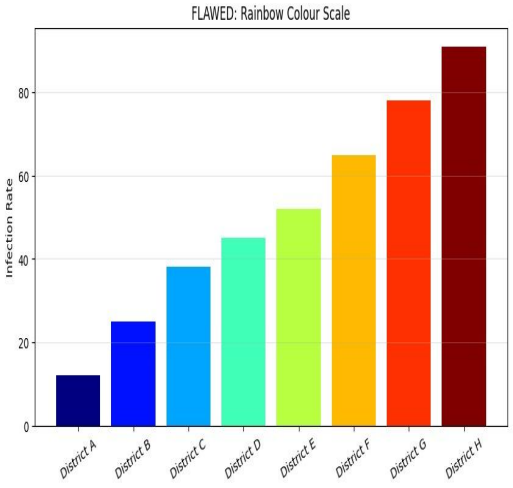
Conclusion

The corrected zero-baseline chart **strengthens the startup's story rather than weakening it**. Although it removes visual exaggeration, it demonstrates that the growth is genuinely

Q4. COVID Dashboard Color Scale

Aim

To evaluate the use of rainbow/jet and red-green color scales in a public health dashboard based on perceptual accuracy, accessibility, and public trust.



Problem Statement

A city COVID dashboard uses:

1. A rainbow/jet color scale for infection rates.
2. Red-green colors to represent safe and high-risk areas.

Both designs are evaluated for possible communication problems.

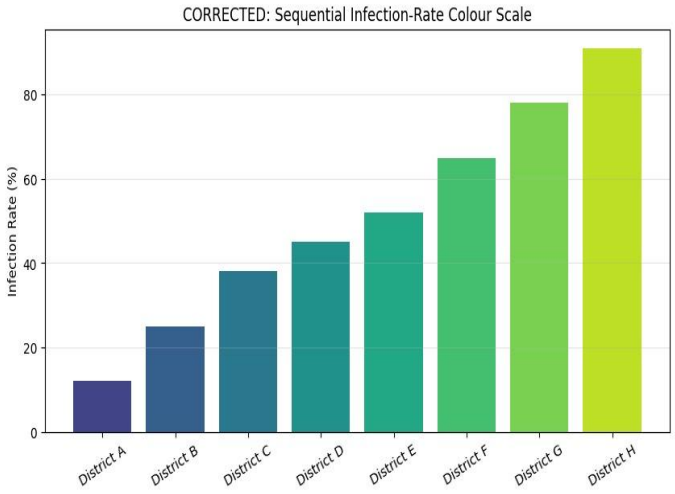
Dataset

District	Infection Rate per 10,000	Risk Category
A	15	Low
B	35	Moderate
C	60	High
D	85	Very High
E	110	Critical

Code

```
import matplotlib.pyplot as plt

import numpy as np


districts = ["A", "B", "C", "D", "E"]

infection_rate = [15, 35, 60, 85, 110]


# Rainbow/jet color scale
```

```
plt.figure(figsize=(8, 5))
plt.bar(districts, infection_rate,
        color=plt.cm.jet(np.linspace(0, 1, len(districts))))

plt.xlabel("District")
plt.ylabel("Infections per 10,000")
plt.title("COVID Infection Rate – Rainbow Scale")
plt.show()
```




```
# Alternative sequential scale
plt.figure(figsize=(8, 5))
plt.bar(districts, infection_rate,
        color=plt.cm.viridis(
            np.linspace(0, 1, len(districts))))

plt.xlabel("District")
plt.ylabel("Infections per 10,000")
plt.title("COVID Infection Rate – Accessible Sequential Scale")
plt.show()
```

Result

The rainbow/jet scale introduces several unnecessary color transitions. These transitions can make numerical differences appear larger or smaller than they actually are.

The red-green scheme has a major accessibility problem because people with certain forms of color-vision deficiency may have difficulty distinguishing red from green.

Ranking of Harm

1. Red-Green – More severe harm

Red-green is particularly problematic because it can completely prevent some users from distinguishing the two intended categories. During a health emergency, this could result in an incorrect understanding of risk.

.

Conclusion

Both color schemes should be replaced. Red-green has the greater accessibility risk, while rainbow/jet has significant perceptual problems. A colorblind-friendly sequential scale combined with labels would provide a more accurate and trustworthy COVID dashboard.

Q5. Visualization of 50,000 GPS Pings

Aim

To compare a plain scatter plot, 2D density contour map, and neighborhood-level bubble map for visualizing the spatial concentration of 50,000 ride-hailing GPS demand points.

Problem Statement

A ride-hailing company needs to visualize where demand is concentrated across a city. Three visualization techniques are considered:

- **A:** Plain scatter plot.
- **B:** 2D density contour map.
- **C:** Direct-area bubble map aggregated by neighborhood.

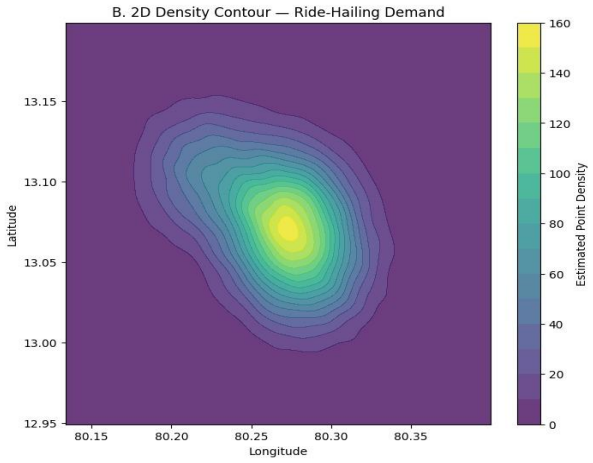
Dataset

A simulated dataset of **50,000 GPS pings** is generated using several demand centers.

Area	Approximate GPS Pings	Demand Level
Downtown	12,000	Very High
Airport	9,000	High
Business District	8,000	High
Residential Area 1	6,000	Moderate
Residential Area 2	5,000	Moderate
Other Areas	10,000	Low/Moderate
Total	**50,000**	—

Code

```
import numpy as np

import matplotlib.pyplot as plt
```




```
np.random.seed(42)

n = 50000

# Simulated city GPS locations
centers = [
    (20, 20),
    (50, 70),
    (70, 30),
    (40, 45),
    (80, 80)
]

weights = [0.25, 0.20, 0.18, 0.17, 0.20]

x = []
y = []

for (cx, cy), w in zip(centers, weights):

    count = int(n * w)

    x.extend(np.random.normal(cx, 5, count))
    y.extend(np.random.normal(cy, 5, count))

x = np.array(x)
y = np.array(y)
```

```
# A. Plain scatter plot
plt.figure(figsize=(7, 6))
plt.scatter(x, y, s=2, alpha=0.3)
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("A. Plain Scatter Plot – 50,000 GPS Pings")
plt.show()

# B. 2D density contour
plt.figure(figsize=(7, 6))
plt.hist2d(x, y, bins=50)
plt.colorbar(label="Number of GPS Pings")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("B. 2D Density Map")
plt.show()

# C. Bubble-style aggregation
bins = 10
hist, xedges, yedges = np.histogram2d(x, y, bins=bins)

xc = (xedges[:-1] + xedges[1:]) / 2
yc = (yedges[:-1] + yedges[1:]) / 2

X, Y = np.meshgrid(xc, yc)

plt.figure(figsize=(7, 6))
plt.scatter(X.flatten(), Y.flatten(),
            s=hist.T.flatten() * 0.5,
```

```
        alpha=0.6)


plt.xlabel("Longitude")

plt.ylabel("Latitude")

plt.title("C. Aggregated Bubble Map")

plt.show()
```

Result

A. Plain Scatter Plot

The scatter plot displays individual GPS observations. It preserves the raw data and can reveal individual locations.

However, with **50,000 points**, substantial overplotting occurs. Areas with many observations become visually crowded, making it difficult to determine exact density.

B. 2D Density Contour/Heat Map

The density visualization aggregates observations into spatial regions and clearly identifies high-demand and low-demand areas.

It reduces overplotting and is effective for identifying demand hotspots.

C. Direct-Area Bubble Map

The bubble map aggregates demand and represents the magnitude of observations through bubble size.

It is relatively easy for managers to interpret and can work well when the analysis is performed at neighborhood level.